

Part 3 分页存储管理

3.1 分页存储基本原理

0	A.0
1	A.1
2	A.2
3	A.3
4	B.0
5	B.1
6	B.2
7	C.0
8	C.1
9	C.2
10	C.3
11	
12	
13	
14	

0	A.0
1	A.1
2	A.2
3	A.3
4	
5	
6	
7	C.0
8	C.1
9	C.2
10	C.3
11	
12	
13	
14	

0	A.0
1	A.1
2	A.2
3	A.3
4	D.0
5	D.1
6	D.2
7	C.0
8	C.1
9	C.2
10	C.3
11	D.3
12	D.4
13	
14	

Part 3 分页存储管理

3.2 地址结构

- 地址结构与数对（页号，页内位移）的形成

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	1	0	1	1	1	0	1	1	1	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	1	0	1	1	1	0	1	1	1	0	0	0

← 页号 → ← 页内地址 →

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	1	0	1	1	1	0	1	1	1	0	0	0

← 页号 → ← 页内地址 →

Part 3 分页存储管理

3.3 数据结构

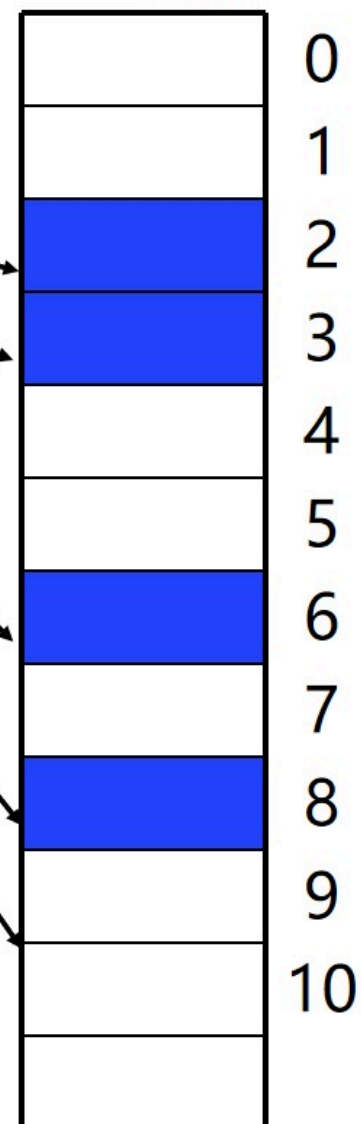
用户程序

0页
1页
2页
3页
4页
5页
.....
n页

页表

页号	块号
0页	2
1页	3
2页	6
3页	8
4页	9
5页	
...	...
n页	

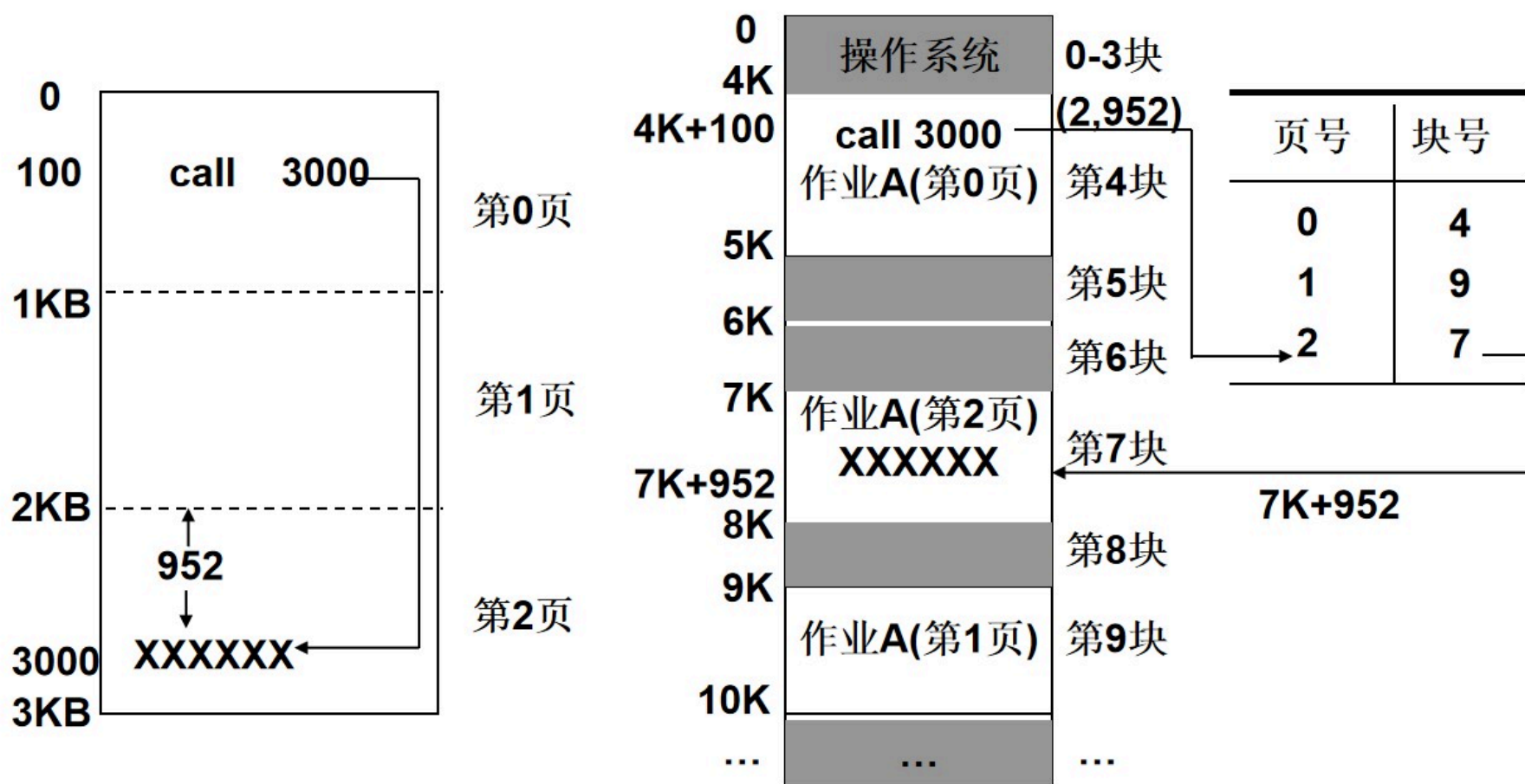
内存



Part 3 分页存储管理

3.4 地址变换

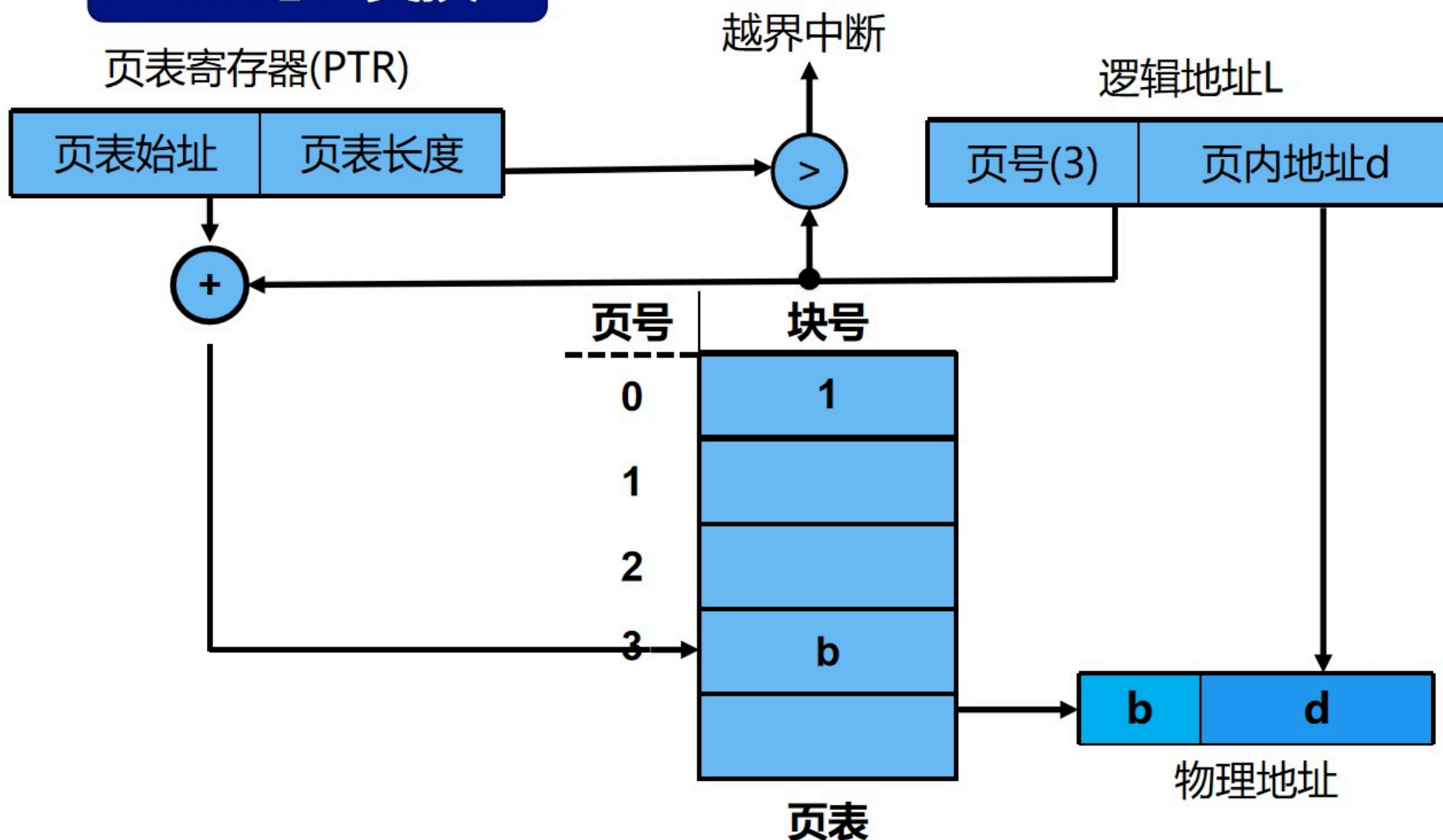
分页存储管理中程序的执行



Part 3 分页存储管理

3.4 地址变换

基本的地址变换机构



3.4 地址变换

地址变换例题1

- 某分页存储管理系统的每页大小2KB，内存为256KB。假定某时刻一用户页表如下：
- 逻辑地址053CH（十六进制）和142FH（十六进制）所对应的物理地址是什么？

页号	物理块号
0	5
1	0
2	9
3	7

3.4 地址变换

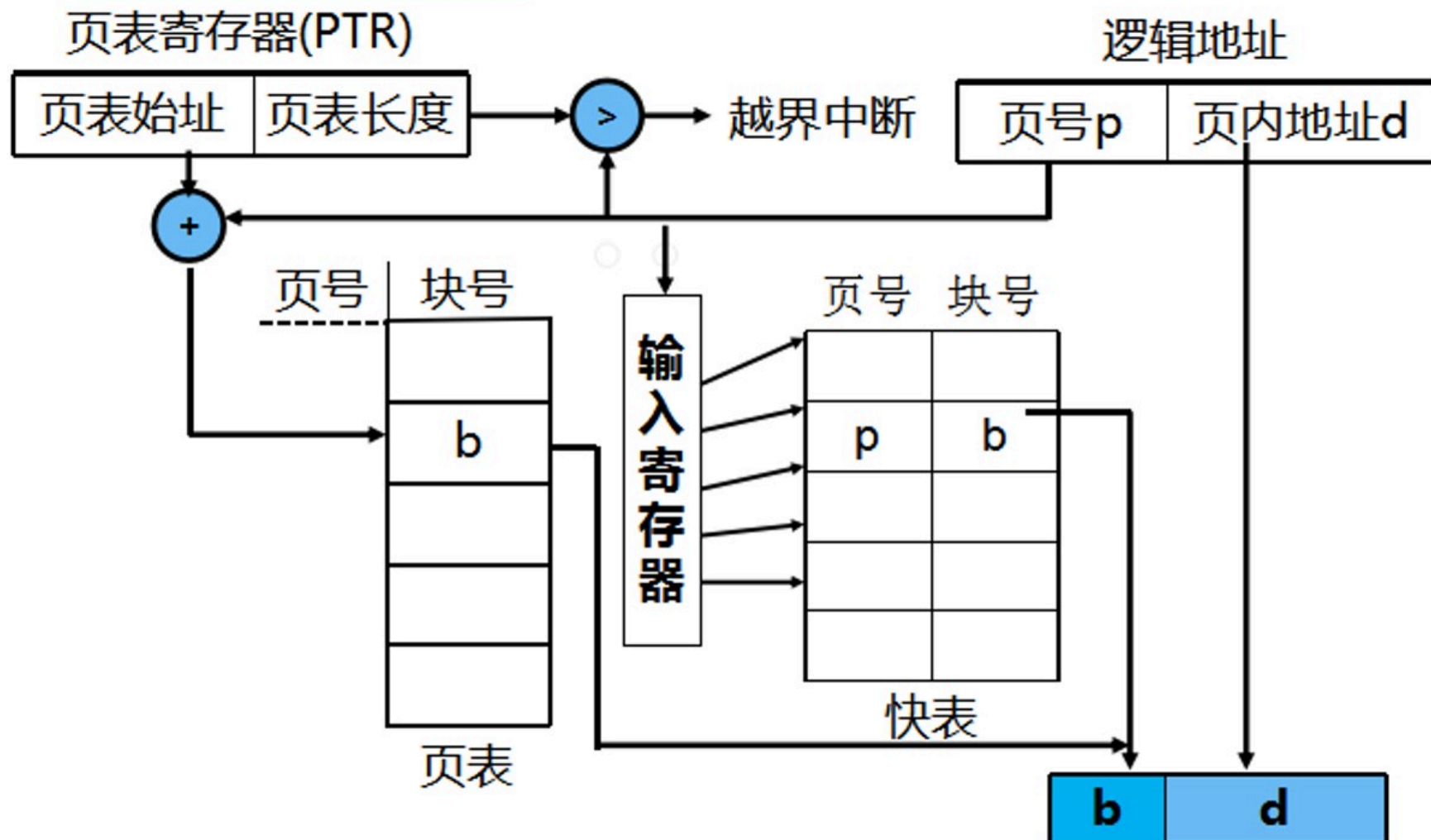
特点

- 增加了指令执行时间：页表在内存，一条指令执行至少需要访问两次内存。
- 缺点：指令执行时间延长

Part 3 分页存储管理

3.5 相关技术

1.快表 具有快表的地址变换机构



3.5 相关技术

2.内存有效访问时间EAT

- ❖ 访问一次内存的时间为 t ，快表命中率为 a ，快表访问一次时间为 λ
- ❖ 基本分页存储管理 $EAT = t + t = 2t$
- ❖ 有快表的分页存储管理方式
- ❖ $EAT = a \times \lambda + (t + \lambda) \times (1 - a) + t = 2t + \lambda - t \times a$

下列内存管理方式中，会产生内碎片的是（ ）。

- ☒ A 单一连续分区 ✓
- ☒ B 固定分区 ✓
- ☒ C 分页 ✓
- ☐ D 可变分区 ✗

关于分页存储管理方式描述不正确的是（ ）。

- ☐ A 分页是将内存等分后程序也分成同样大小分开存储 ✓
- ☒ B 分页的优点是没有任何内存碎片 ✗
- ☒ C 分页不会影响进程执行时间 ✗
- ☐ D 分页的地址转换需要额外的硬件支持 ✓

单选题 1分

分页方式中，页面大小为4KB，某进程逻辑地址空间大小为1GB，该进程的一页大小占（ ）位

A 20

B 8

~~C 11~~

D 12

关于进程页表描述不正确的是（ ）。

☒ A

每个进程一张页表，描述进程各页面的存储情况

☒ B

页表的大小和进程大小有关

☐ C

页表属于进程的地址空间

☒ D

进程的地址转换过程需要访问页表

单选题 1分

有一个分页系统，访问一次内存需要0.2us，系统中有快表，快表命中率为90%，假定查快表花费的时间为0.01us，内存的有效访问时间是 () us。

- A 0.4
- B 0.41
- C 0.21
- D 0.23
- $+ 0.01 + 0.1 \times 0.2 = 0.23$
 $+ 0.2$

3.5 相关技术

3.两级或多级页表



现代的大多数计算机系统，都支持非常大的逻辑地址空间。在这样的环境下，页表就变得非常大，要占用相当大的内存空间。



解决办法：

- 对页表所需要的内存空间，采用离散分配方式
- 部分页表调入内存



页表结构

- 两级页表
- 多级页表
- 反置页表

Part 3 分页存储管理

3.5 相关技术

两级页表

例子 (32位逻辑地址空间)



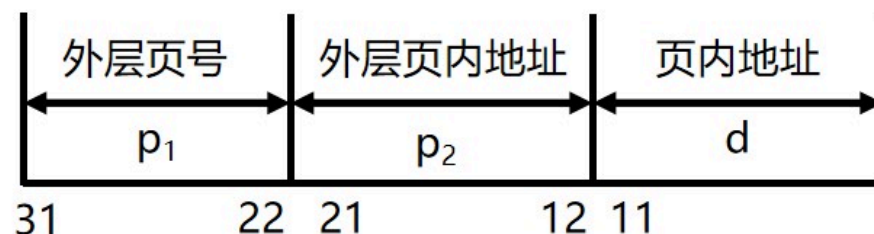
页面大小为4KB时 (12位), 若采用一级页表结构, 应具有20位的页号, 即页表项应有1M个;



在采用两级页表结构时, 再对页表进行分页, 使每页中包含 2^{10} 个页表项, 最多允许有 2^{10} 个页表分页。



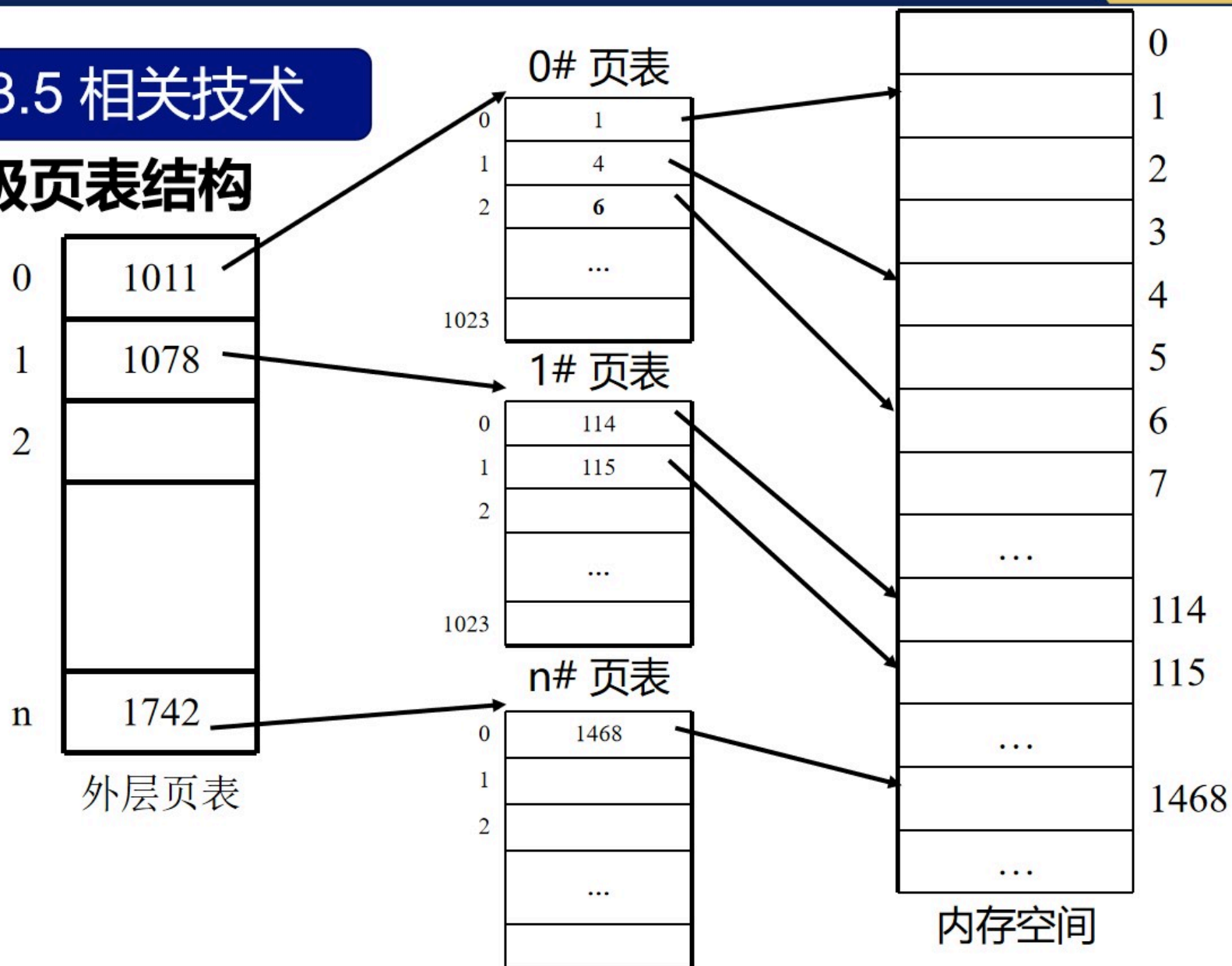
地址变换机构上, 增设一个外层页表寄存器



Part 3 分页存储管理

3.5 相关技术

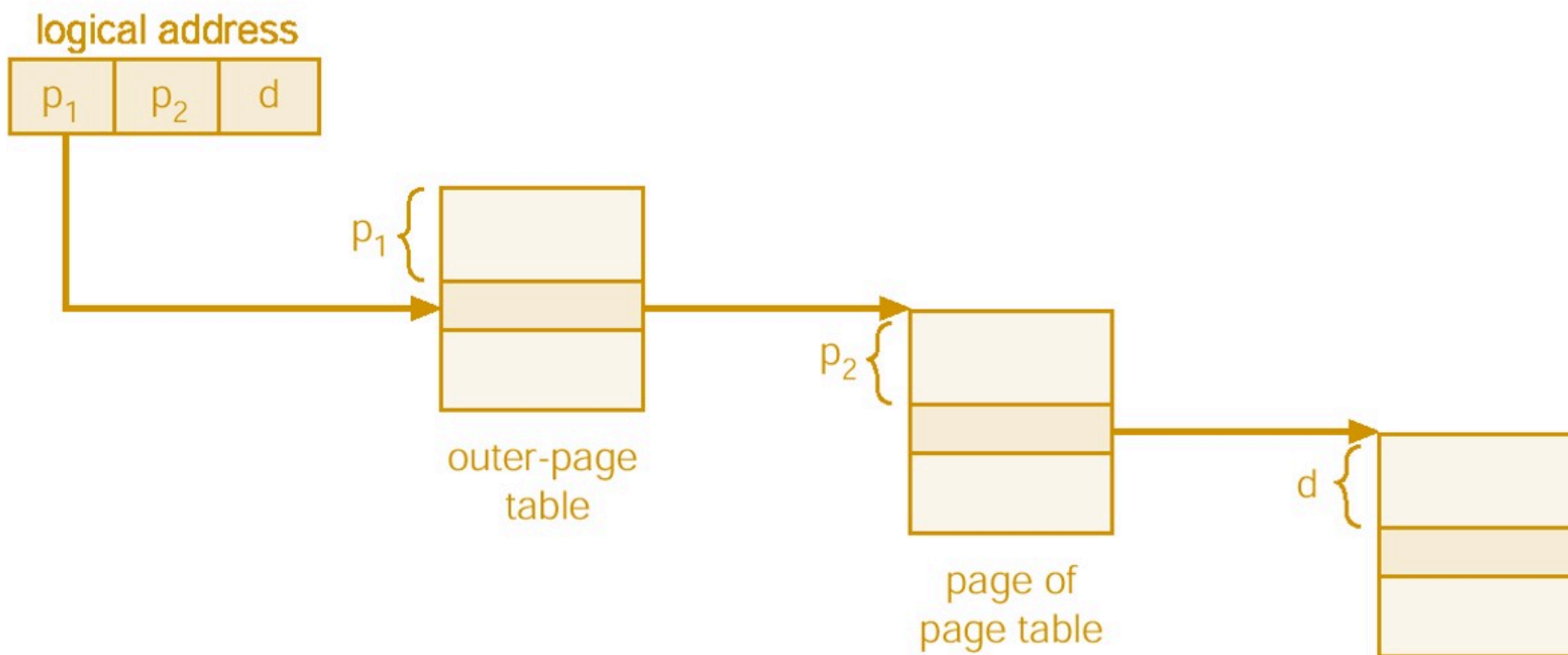
两级页表结构



Part 3 分页存储管理

3.5 相关技术

两级页表结构



Part 3 分页存储管理

3.5 相关技术

例题

某计算机采用二级页表的分页存储管理方式，按字节编址，页大小为 2^{10} 字节，页表项大小为2字节，逻辑地址结构为

页目录号	页号	页内偏移量
------	----	-------

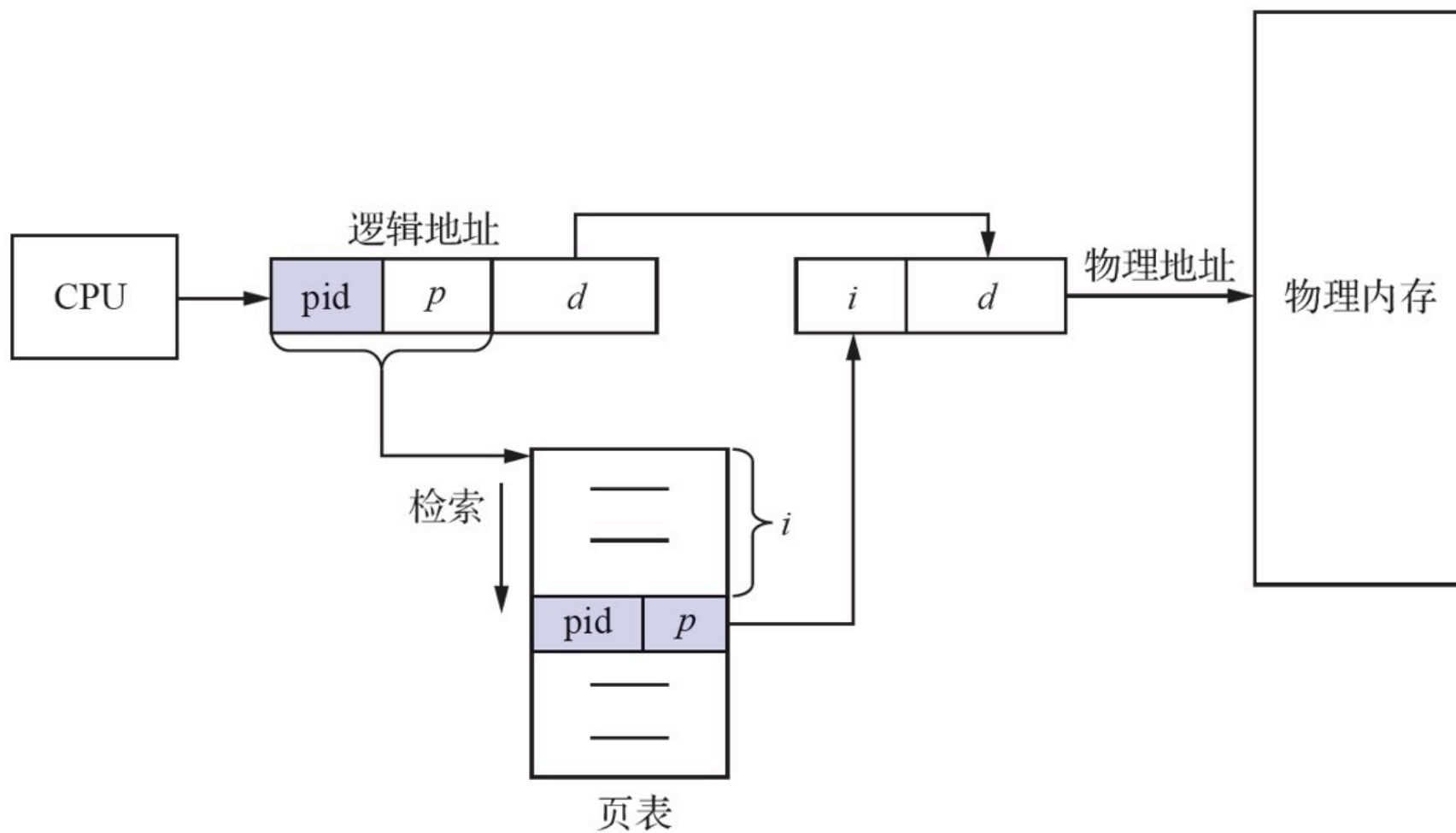
逻辑地址空间大小为 2^{16} 页，则表示整个逻辑地址空间的页目录表中包含表项的个数至少是

(A)64 (B)128 (C)256 (D)512

Part 3 分页存储管理

3.5 相关技术

反置页表



3.6 分页特点

●特点:

- ❖一维编址
- ❖两次访存
- ❖连接方式 (静态)

●优点:

- ❖一个程序不必连续存放。
- ❖没有外碎片, 每个内碎片不超过页大小, 每个程序平均浪费半页空间。
- ❖空间利用率最高

●缺点:

- ❖页的大小与程序的逻辑大小无关
- ❖程序的共享与保护难于实现
- ❖进程全部装入内存。

Part 4 分段存储管理



学习目标

- 能够理解分段存储管理的原理
- 能够完成分段的地址变换
- 理解并能分析分页与分段的区别

Part 4 分段存储管理

4.1 分段基本原理

➤ 引入目的: 为了满足用户（程序员）在编程和使用上多方面的要求

➤ 一个程序是一些段的集合, 一个段是一个逻辑单位, 如:

main program,
procedure,
function,
local variables, global variables,
common block,
stack,
symbol table, arrays

Part 4 分段存储管理

4.1 分段基本原理

主程序段

call [X] | <E>
(调用X段的入口E)

call [Y] | <F>
(调用Y段的入口F)

load 1,[A] | <G>
(调用数组段A[G])

子程序段X

E: -----

子程序段Y

F: -----

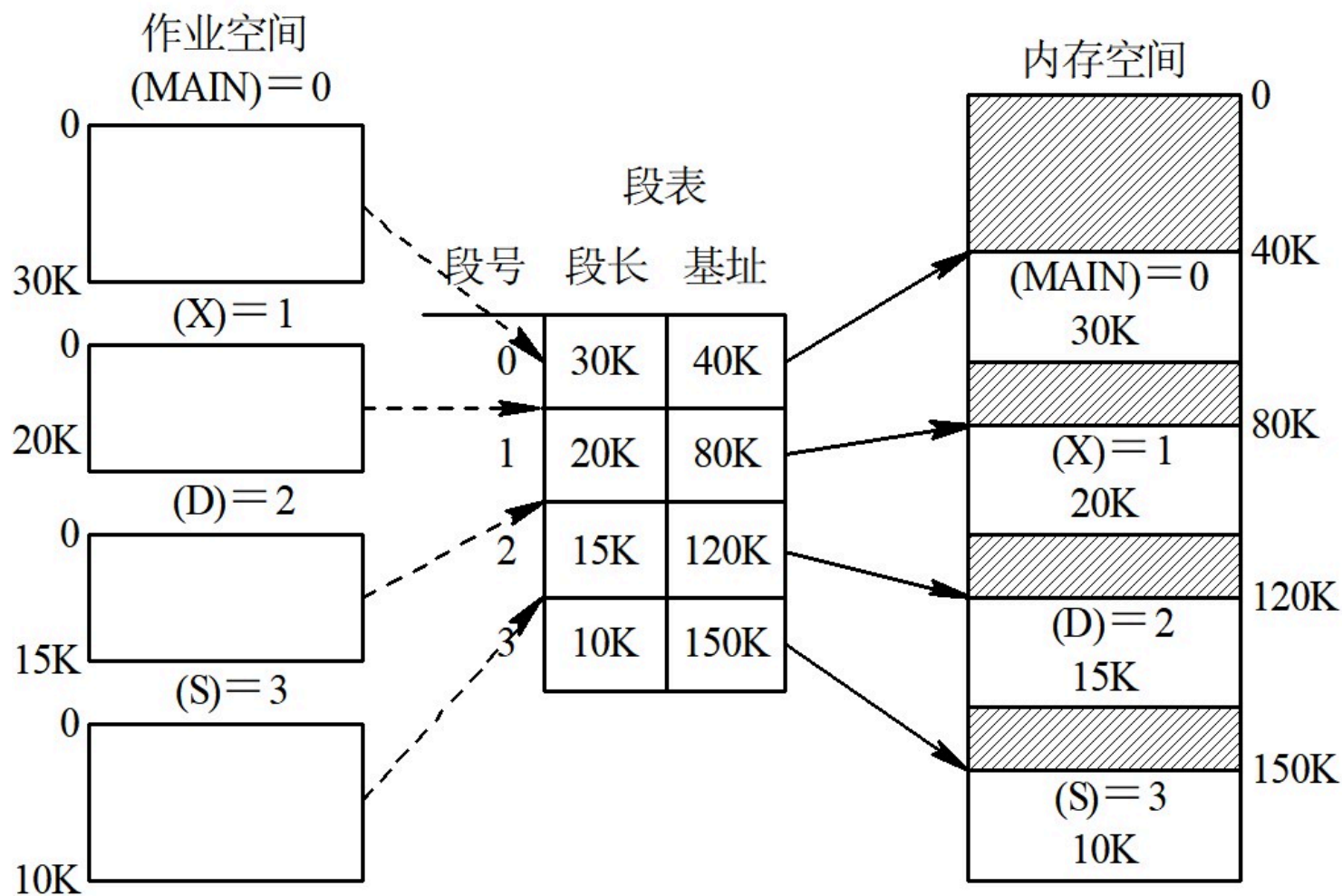
数组段A

G: -----

工作区段

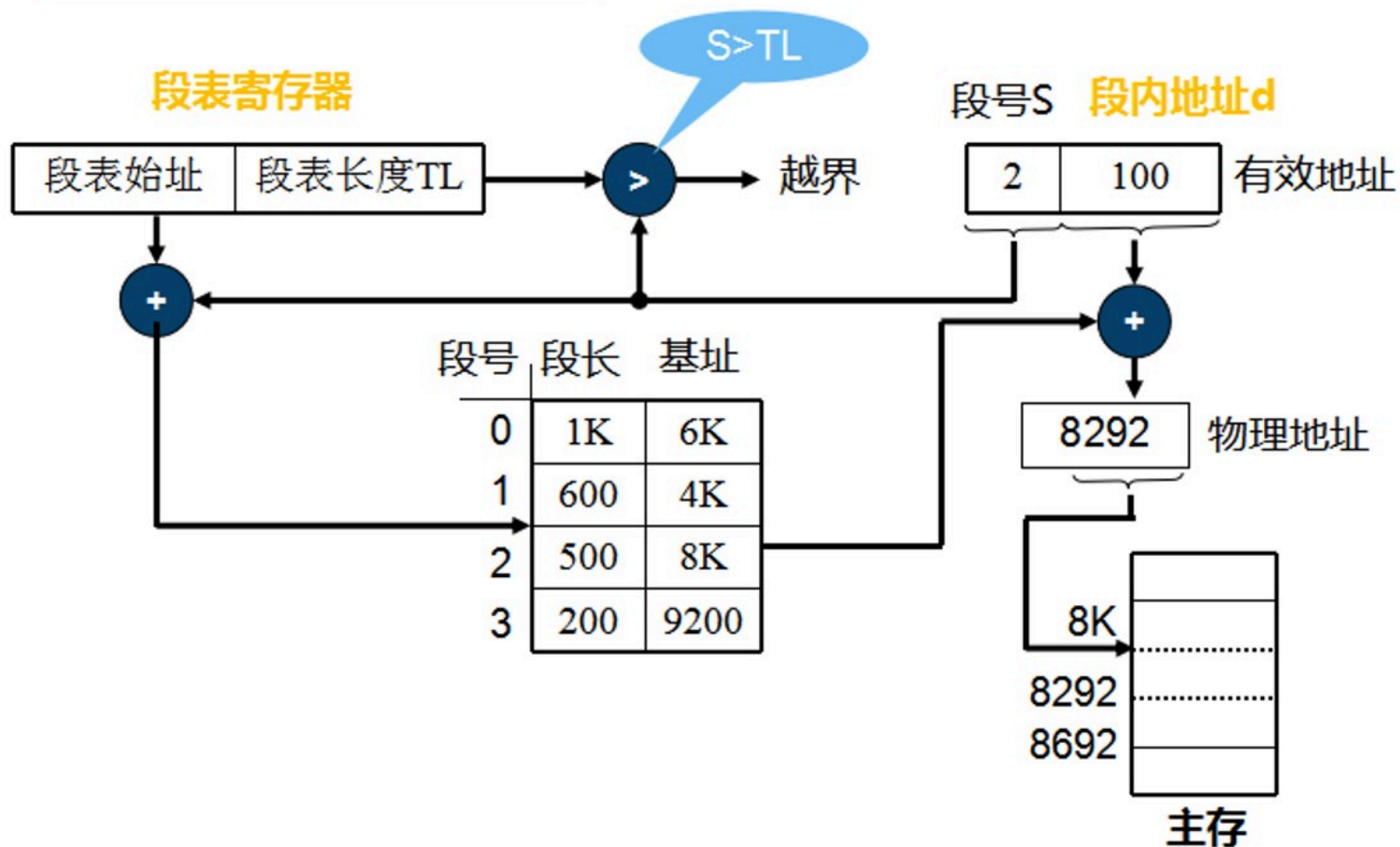
Part 4 分段存储管理

4.2 数据结构



Part 4 分段存储管理

4.3 地址变换



Part 4 分段存储管理

4.3 地址变换

- 在分段存储管理中，有某作业的段表如下：

段号	段长	基址
0	600	219
1	14	2300
2	100	90
3	580	1327
4	96	1954

已知该作业中的6个逻辑地址为：

- (1) [0,430]; (2) [3,400]; (3) [1,10]; (4) [2,2500]; (5) [4,42]; (6) [1,11]。试求其对应的物理地址。

Part 4 分段存储管理

4.4 特点

- **特点:**

- ❖ 二维编址，二次访存
- ❖ 模块可以分别编写和编译
- ❖ 链接方式（动态）

- **优点**

- ❖ 符合程序的结构化设计。便于实现共享和保护。

- **缺点:**

- ❖ 空间利用率、出现外部碎片，碎片的紧凑。
- ❖ 进程全部装入内存。

Part 4 分段存储管理

4.4 分段与分页

- 出发点
- 指令跨页问题
- 页和段大小
- 逻辑地址
- 页表和段表大小

Part 4 分段存储管理

4.4 分段与分页

- **分页是出于系统管理的需要，分段是出于用户应用的需要。**
 - ❖ 一条指令或一个操作数可能会跨越两个页的分界处，而不会跨越两个段的分界处。
- **页大小是系统固定的，而段大小则通常不固定。**
- **逻辑地址表示：**
 - ❖ 分页是一维的，各个模块在链接时必须组织成同一个地址空间；
 - ❖ 分段是二维的，各个模块在链接时可以每个段组织成一个地址空间。
- **通常段比页大，因而段表比页表短，可以缩短查找时间，提高访问速度。**

4.6 段页式存储管理

1. 基本原理



基本原理:

- 分段和分页原理的结合, 即先将用户程序分成若干段, 再把每个段分成若干个页, 并为每个段赋予一个段名。



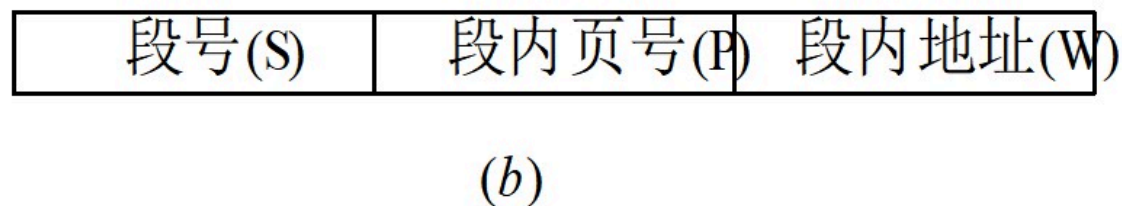
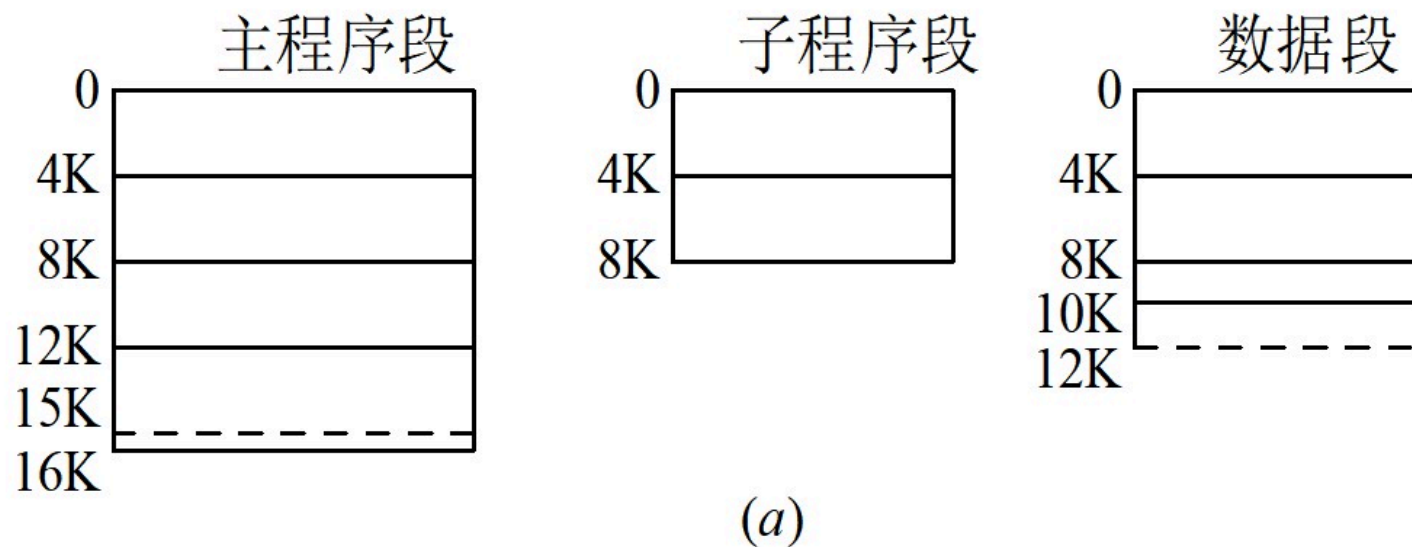
优点:

- 既有分段系统的便于实现、可共享、易于保护、可动态链接;
- 又能像分页系统, 很好地解决内存的外部碎片问题。

Part 4 分段存储管理

4.6 段页式存储管理

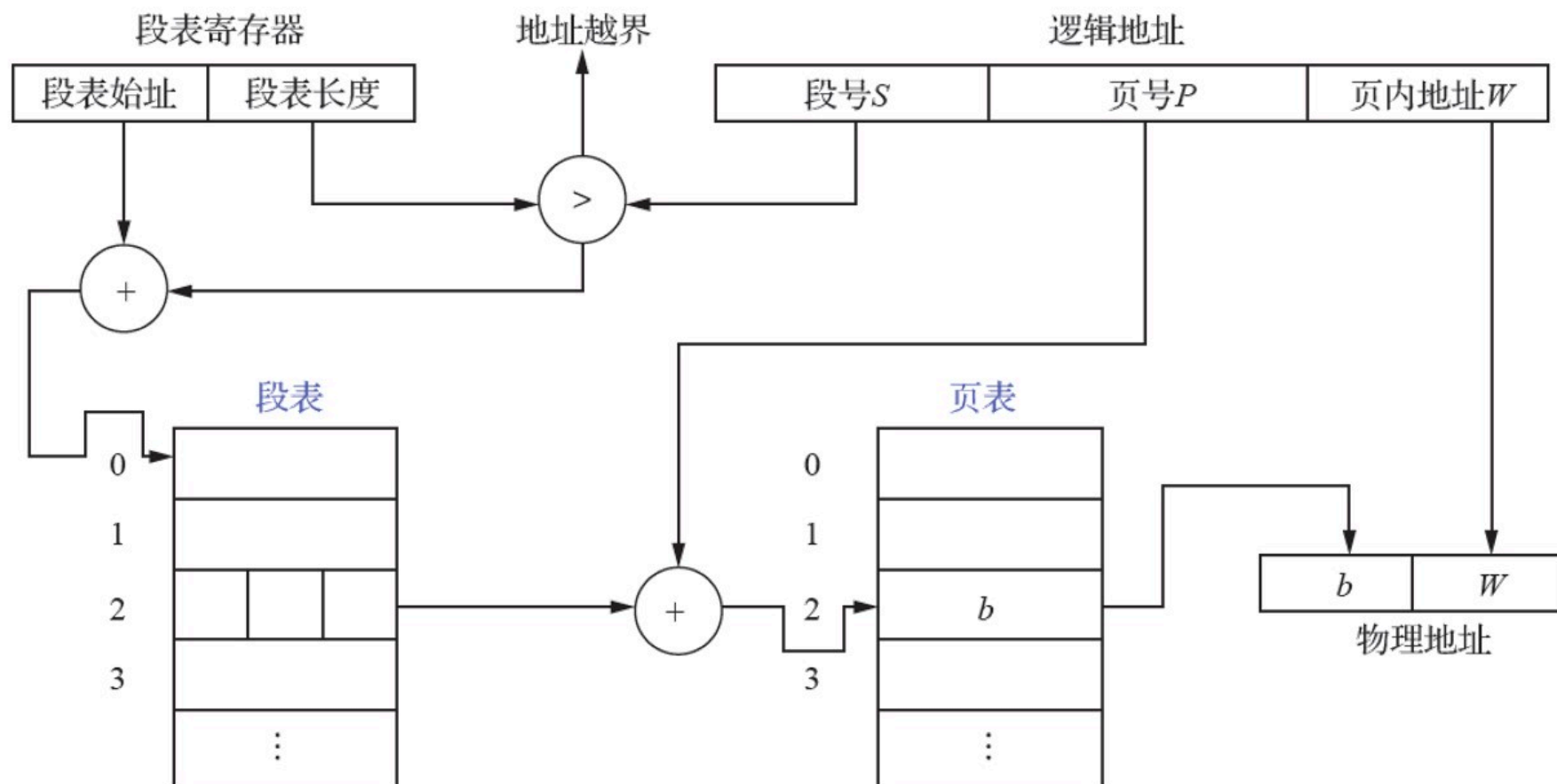
1. 基本原理



Part 4 分段存储管理

4.6 段页式存储管理

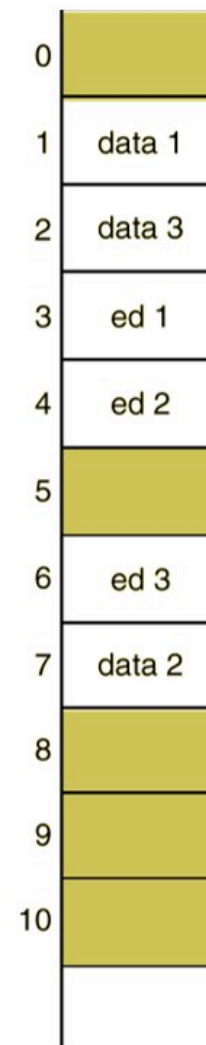
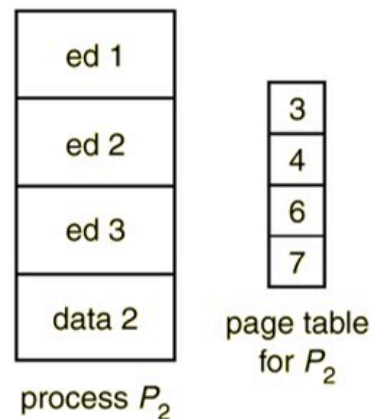
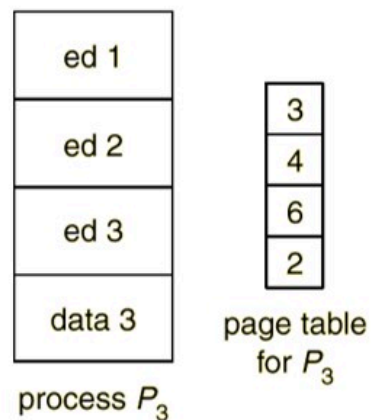
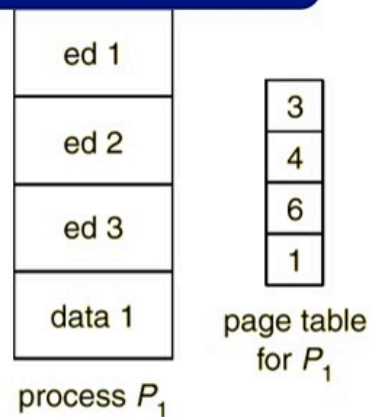
2.地址变换



Part 4 分段存储管理

4.7 信息共享

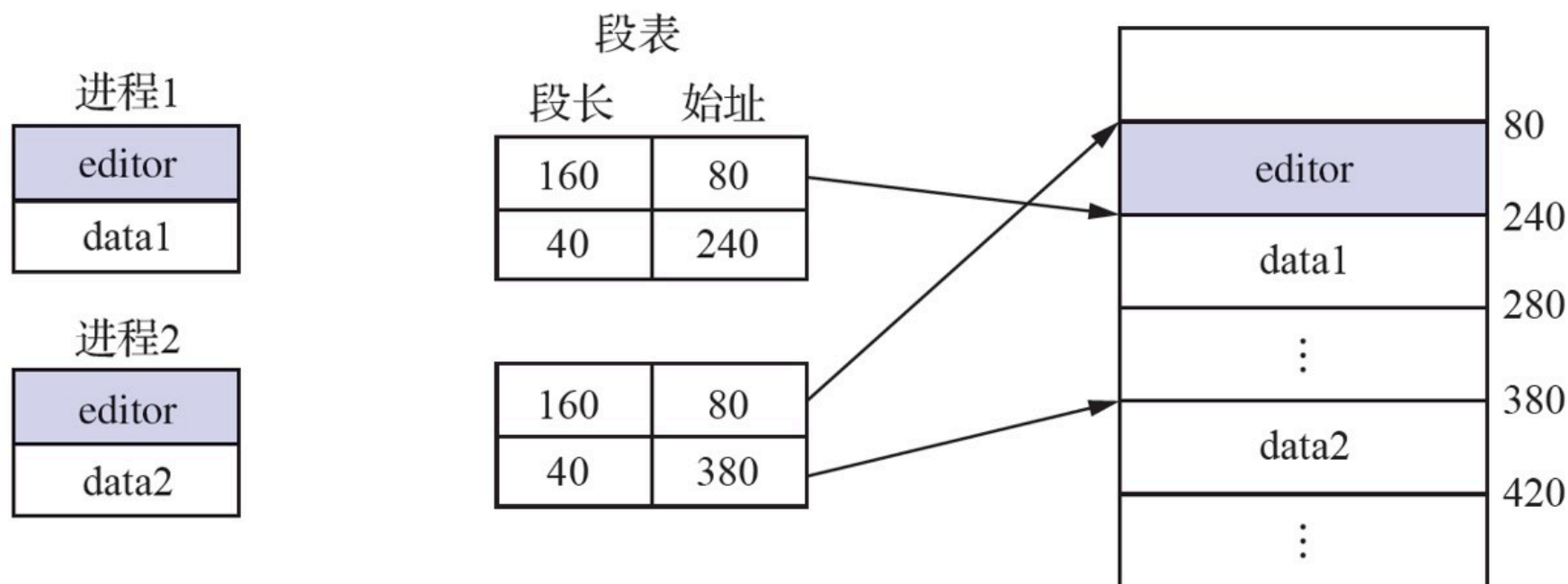
分页共享



Part 4 分段存储管理

4.7 信息共享

分段共享





课后习题

- 3.什么是重定位？为什么要重定位？
- 7.何为链接？链接主要解决什么问题？
- 11.提高内存利用率的途径有哪些？